

Required URLs

- LeetCode Solution:
<https://leetcode.com/submissions/detail/2117808745/>
- LinkedIn Post 1:
https://lnkd.in/p/er_2rJAn
- LinkedIn Post 2:
<https://lnkd.in/p/ey8jGU3R>
- LinkedIn Post 3:
<https://lnkd.in/p/ezEQCiJj>

git squash

السيناريو:

تخيل انك شغال على feature جديدة في ال backend مثلا بتبني ال database schema بتاعت inventory microservice. طول ما انت شغال بتعمل commits صغيرة عشان تحفظ شغلك اول باول فعملت حوالي 5 او 6 commits كانوا كده:

```
init models  
fix typo in column name  
wip  
testing alembic migration  
finally working
```

التوضيح:

لو عملت merge لل branch ده علي ال main بالمنظر ده هتلاقي ال commit history بقت زحمة و مش منظمة و كمان فيها commits ملهاش لازمة. ده بيصعب الدنيا لو احتجت ترجع لل history و تصلح bug معينة مثلا بعدين.

git squash

هنا يبجي دور ال git squash. قبل ما تفتح pull request مثلا بتعمل interactive rebase و ت squash (تدمج) كل ال commits العشوائية دي في commit واحدة بس و برسالة واضحة زي:

```
feat: setup inventory database models and migrations
```

بكده تقدر تضمن ان ال history بتاع ال repository يكون نظيف و مقري و ان كل commit تبقا بتمثل feature او bug fix كاملة على بعضها.

git merge vs. git rebase

السيناريو:

تخيل انك فاتح branch جديدة بتعمل فيها integration ل RabbitMQ أو بتبني service جديدة. قعدت شغال يومين و في نفس الوقت زمالك خلصوا شغلهم و عملوا merge لتعديلات جديدة علي ال main branch. دلوقتي ال branch بتاعك متأخر عن ال main و محتاج تسحب التعديلات دي عندك عشان تتأكد ان شغلك مش هيجيب conflicts قبل ما تفتح PR.

التوضيح:

قدامك طريقين:

1. لو استخدمت git merge main و انت جوا ال branch بتاعك فا Git هيدمج التعديلات الجديدة و يعمل commit مخصوص اسمها merge commit.

- بنستخدمه لما نكون عايزين نحافظ علي ال timeline الحقيقي بتاع ال commits بحيث كل حاجة تتسجل زي ما حصلت بالطبط.
- لكن: لو التيم كبير و كل شوية حد يعمل merge فا ال history بيبقا متفرع جدا و معقد.

git merge vs. git rebase

2. انما لو استخدمت git rebase main فا Git هياخد ال commits بتاعتك يركنها مؤقتا و يسحب احدث نسخة من ال main و بعدين يرجع يكتب ال commits بتاعتك فوقها بالترتيب.

- بنستخدمه عشان نبين ان ال history حصل في خط مستقيم كأنك لسه كاتب الكود بتاعك حالا فوق احدث شغل لزمالك و ده ببسهل جدا قراءة ال history بعدين.
- بس خلي بالك في تحذير مهم: ال rebase بيعمل rewrite لل history عشان كده بنستخدمه بس طول ما ال commits لسه local علي جهازك و معمלתهاش push.

git clean

السيناريو:

تخيل إنك بتعمل debugging لميكروسيرفس زي ال inventory service. وإنت بتجرب اتكربت عندك ملفات كتير عشوائية زي ملفات logs أو ملفات configs مؤقتة أو فولدرات pycache لسه متضافتش في ال .gitignore. إنت عايز ترجع ال working directory لحالته النظيفة وتمسح الزحمة دي. المشكلة إن git restore أو git checkout بيمسح التعديلات في ال tracked files بس ومش بيلمس الملفات الجديدة غير المتتبعة.

التوضيح:

هنا ببيجي دور git clean. الكوماند ده وظيفته يمسح أي ملفات أو فولدرات غير متتبعة من ال repo تماماً. في الشغل غالباً بنستخدمه بالصيغة دي: git clean -fd

- حرف ال f اختصار force: عشان Git يمسح الملفات

- حرف ال d اختصار directories: عشان يمسح الفولدرات غير المتتبعة كمان

بنستخدمه لما نحب نعمل reset كامل ونمسح أي ملفات اتكريتت بالغلط أو ملهاش لازمة أثناء ال development عشان نرجع لبيئة نظيفة.

git cherry-pick

السيناريو:

تخيل شغال على feature في ال branch بتاعك مثلا integration ل Redis وفي نفس الوقت زميلك حل critical bug في ال branch بتاعه. إنت محتاج ال fix ده ضروري لشغلك بس ال branch بتاع زميلك فيه شغل كتير لسه مش كامل ومليان bugs فمش هينفع تعمل merge لل branch ده دلوقتي.

التوضيح:

هنا بييجي دور git cherry-pick. بدل ما تسحب كل الشغل اللي لسه مش خلصان بتاع زميلك إنت محتاج ال fix ده بس:

- خذ ال commit hash بتاع ال fix ده
- روح لل branch بتاعك واكتب `<git cherry-pick <commit-hash`
- كده ال commit دي هتنتقل عندك لوحدها من غير أي كود ثاني مش محتاجه

git grep

السيناريو:

تخيل انك بتعمل refactoring لل codebase. قررت تغيير اسم function بتكلم ال database او بتدور على API endpoint قديم عشان تمسحه. ال repository كبير وفيه ملفات كتير ولو دورت بالبحث العادي او ب grep العادي في اللينكس ممكن ياخذ وقت ويطلعك نتايج من جوه ال virtual environment او ملفات ال logs او الفولدرات اللي متجاهلة في ال .gitignore.

التوضيح:

هنا بنستخدم git grep. الكوماند ده بيدور على ال text او ال pattern اللي انت عايزه جوه ال tracked files بس يعني بيتجاهل اوتوماتيك اي حاجة موجودة في ال gitignore وده بيخليه سريع في البحث و دقيق. تقدر تكتب ببساطة "git grep" def get_inventory_item عشان يجيبلك كل الاماكن اللي ال function دي مكتوبة فيها وهيقولك اسم الملف ورقم السطر. بنستخدمه لما نكون عايزين ندور على text معين او error message او variable في المشروع كله بسرعة جدا من جوه ال terminal او حتى لو عايزين ندور في branch ثاني من غير ما نعمله checkout.

git blame

السيناريو:

تخيل انك بتقرا كود في API endpoint معينة او بتراجع كويري SQL معقدة جدا جوه ال repository ومش فاهم اللوجيك بتاعها اتكتب بالطريقة دي ليه او لقيت سطر كود هو اللي مسبب bug في ال .production

التوضيح:

بتكتب git blame filename. الكوماند ده بيطلعك الملف سطر سطر وجنب كل سطر بيكتبلك اسم الشخص اللي عمله commit والتاريخ وال commit hash. بنستخدمه عشان نعرف مين اخر حد عدل في السطر ده فنروح نساله عن اللوجيك ورا الكود بدل ما نخمن.

git bisect

السيناريو:

تخيل ان ال Redis caching كان شغال زي الفل من اسبوع بس النهاردة اكتشفتموا انه مش شغال وبيوقع ال requests. المشكلة ان في خلال الاسبوع ده التيم عمل اكتر من 50 commit ومحدث عارف ايه التغيير بالظبط اللي بوظ الدنيا.

التوضيح:

git bisect بيعمل Binary Search في ال commit history. بتقوله ان الكود دلوقتي bad ومن 50 commit كان good. هو اوتوماتيك بيعمل checkout لل commit اللي في النص بالظبط ويقولك جرب.. لو شغال تقوله good فيستبعد النص القديم ولو بايظ تقوله bad. بيفضل يقسم ال commits للنص لحد ما يحط ايده على ال commit اللي دخل ال bug بالظبط في ثواني.

git shortlog

السيناريو:

تخيل انكم بتقفلوا Sprint و هتعملوا Release جديدة لل microservice بتاعتكم وال Team Lead او ال Project Manager طلب منك ملخص سريع لكل اللي اتعمل في ال release دي ومين عمل ايه.

التوضيح:

بدل ما تقرا git log الطويل والممل git shortlog بيجمعلك ال commits كلها ويعملها Grouping باسماء المبرمجين. بيطلعلك كل مبرمج اسمه ايه وعمل كام commit وعناوين ال commits دي ايه. ممتاز جدا لل changelogs وتوثيق الشغل.

git prune

السيناريو:

بعد شهور من الشغل على المشروع وعملت مئات ال branches ومسحتها وعملت rebase كثير و force push ال repository مساحته بتبدأ تكبر على الفاضي بسبب حاجة اسمها dangling objects دي عبارة عن commits او ملفات Git كان حافظها بس مبقاش ليها اي طريق او branch يوصلها.

التوضيح:

git prune بي مسح اي objects ملهاش اي reference او مش مربوطة باي branch حالي. هو بيشبه ال Garbage Collector في البرمجة بنستخدمه عشان ننظف الداتا الميتة ونوفر مساحة وغالبا بيشتغل .git gc اوتوماتيك مع كوماندا .git gc.

git worktree

السيناريو:

git worktree

تخيل أنك شغال في ال branch بتاعك بتعمل refactor كبير لل Database Models. الكود منكسر وملفات كتير متعدلة ومش جاهزة تتعمل commit وحتى ال stash هيبقى زحمة وممكن يعمل مشاكل. فجاه جالك تليفون في مشكلة في ال Payment Service على ال main branch ومحتاجينك تحلها فوراً.

التوضيح:

بدل ما تيوظ ال workspace بتاعتك بتستخدم git worktree add. ده بيكرينلك فولدر جديد تماماً على جهازك مربوط بنفس ال repository ويعمل checkout لل branch اللي انت عايزه في الفولدر ده. تقدر تفتح ال hotfix في ال editor وتحل المشكلة وتعمل PR وبعدين تقفل الفولدر وترجع للفولدر الاصلي بتاعك. تلاقى كودك زي ما سيبته بالظبط.

git verify-commit

السيناريو:

لو انت شغال في شركة ال Security فيها عالي جداً او بتساهم في Open Source project كبير. اي حد ممكن يغير اسمه وايميله في ال git config ويعمل commit كانه Linus Torvalds او كانه ال CTO بتاع شركتك.

التوضيح:

عشان نمنع الانتحال ده المبرمجين بيوقعوا ال commits بتاعتهم ب GPG Keys. بنستخدم git verify-commit عشان نتأكد ان ال commit جاية فعلاً من الشخص صاحب ال GPG Key الموثوق بيه.

git filter-repo

السيناريو:

git filter-repo

زميلك رفع ملف env بالغلط فيه باسورد الداتا بيز بتاع ال production مسحه في ال commit اللي بعده بس الباسورد لسه موجود في ال history واي حد يقدر يشوفه او عندكم monolith كبير وقررتوا تاخذوا فولدر ال inventory وتعملوله repository لوحده عشان يبقى microservice

التوضيح:

git filter-repo هو البديل الحديث والاسرع للكوماند القديم git filter-branch بنستخدمه عشان نمسح ملف معين من ال history كله من اول ما المشروع بدا كانه لم يكن او عشان نقص فولدر معين بكل ال commits بتاعته وننقله في repo جديد هو بيعمل rewrite لل history بالكامل